Atelier Machine Learning

Analyse Comportementale Clientèle Retail

COMPTE RENDU

Atelier Pratique : E-commerce de Cadeaux

Préparé par Mohamed Amine Gharbi

Année Universitaire : 2025–2026

Table des matières

1 Introduction et Contexte **2**
1.1 Objectifs du projet . . . 2

2 Configuration de l'Environnement de Travail **2**
2.1 Structure du projet . . . 2
2.2 Dépendances principales . . . 2

3 Exploration et Préparation des Données **3**
3.1 Description du dataset . . . 3
3.1.1 Features numériques (sélection) . . . 3
3.1.2 Features catégorielles (sélection) . . . 3
3.2 Problèmes de qualité et traitement . . . 3
3.3 Pipeline de prétraitement . . . 4
3.3.1 Parsing des features brutes . . . 4
3.3.2 Feature Engineering . . . 4
3.3.3 Split Train/Test et normalisation . . . 4

4 Modélisation Avancée **4**
4.1 Module 1 — Analyse en Composantes Principales (ACP) . . . 4
4.2 Module 2 — Clustering K-Means . . . 5
4.2.1 Sélection du nombre optimal de clusters . . . 5
4.2.2 Visualisation des 4 clusters . . . 5
4.3 Module 3 — Classification du Churn . . . 6
4.3.1 Modèles entraînés . . . 6
4.3.2 Résultats de classification . . . 6
4.3.3 Importance des features . . . 7
4.4 Module 4 — Régression de MonetaryTotal . . . 7

5 Déploiement — Application Web Flask **8**
5.1 Routes de l'application . . . 8
5.2 Chargement des modèles et API REST . . . 8

6 Discussion et Recommandations **9**
6.1 Analyse critique . . . 9
6.2 Recommandations métier . . . 9

7 Conclusion **9**

1 Introduction et Contexte

Dans le cadre de l'atelier de Machine Learning du module GI2, nous avons réalisé un projet complet d'analyse comportementale de la clientèle d'une entreprise e-commerce spécialisée dans les cadeaux. L'objectif principal est de mettre en œuvre la chaîne complète de traitement en data science, depuis l'exploration des données jusqu'au déploiement d'une interface utilisateur.

1.1 Objectifs du projet

Ce projet couvre l'ensemble du pipeline ML à travers quatre tâches complémentaires :

- **Segmentation client (Clustering)** — Application de K-Means pour identifier des groupes homogènes de clients.
- **Réduction dimensionnelle (ACP)** — Compression de l'espace de 52 features en conservant au moins 95% de la variance.
- **Classification (Churn)** — Prédiction de la probabilité de départ des clients grâce à Random Forest et Gradient Boosting.
- **Régression (MonetaryTotal)** — Estimation du chiffre d'affaires attendu par client.

2 Configuration de l'Environnement de Travail

2.1 Structure du projet

Arborescence du projet

```
projet_ml_retail/
|-- data/
|   |-- raw/                # Donnees brutes
|   |-- processed/          # Donnees nettoyees
|   `-- train_test/         # X_train, X_test, y_train, y_test
|-- notebooks/              # Notebook Jupyter
|-- src/
|   |-- utils.py
|   |-- preprocessing.py
|   |-- train_model.py
|   `-- predict.py
|-- models/                 # Modeles .joblib
|-- app/
|   |-- app.py
|   `-- templates/
|-- reports/
|-- requirements.txt
`-- README.md
```

2.2 Dépendances principales

Bibliothèque	Usage
`numpy`, `pandas`	Manipulation des données
`scikit-learn`	Modèles ML, prétraitement, métriques
`matplotlib`, `seaborn`	Visualisations
`flask`	Application web de déploiement
`joblib`	Sauvegarde/chargement des modèles

3 Exploration et Préparation des Données

3.1 Description du dataset

Le dataset contient **2 000 clients** décrits par **52 features** issues de transactions réelles et de données comportementales.

3.1.1 Features numériques (sélection)

N°	Feature	Type	Description	Unité
2	Recency	Entier	Jours depuis dernier achat	jours
3	Frequency	Entier	Nombre de commandes	cmd
4	MonetaryTotal	Float	Somme totale dépensée	£
5	MonetaryAvg	Float	Montant moyen par commande	£
13	CustomerTenure	Entier	Durée relation client	jours
27	ReturnRatio	Float	Taux de retour	ratio
32	SupportTickets	Float	Tickets support	–
33	Satisfaction	Float	Note de satisfaction	/5
34	Churn	Binaire	Départ client	0/1

3.1.2 Features catégorielles (sélection)

N°	Feature	Card.	Valeurs	Encodage
35	RFMSegment	4	Champions, Fidèles, Potentiels, Dormants	One-Hot
37	SpendingCat	4	Low, Medium, High, VIP	Ordinal
38	CustomerType	5	Hyperactif, Régulier, Occasionnel...	One-Hot
41	Region	8	UK, Europe, Asie, Autre	One-Hot
43	ChurnRisk	4	Faible, Moyen, Élevé, Critique	Ordinal
47	Gender	3	M, F, Unknown	One-Hot
49	Country	37+	UK, France, Germany...	Target

3.2 Problèmes de qualité et traitement

Type	Feature(s)	Fréq.	Traitement
Valeurs manquantes	Age	30%	KNNImputer (médiane)
Valeurs aberrantes	SupportTickets	5-8%	Remplacé par NaN puis médiane
Valeurs aberrantes	Satisfaction	5-8%	Remplacé par NaN puis médiane
Formats inconsistants	RegistrationDate	100%	`pd.to_datetime`
Feature constante	NewsletterSubscribed	100%	Suppression
Feature brute	LastLoginIP	100%	IP_IsPrivate, IP_FirstOctet
Déséquilibre classes	Churn (≈25%)	Var.	`class_weight='balanced'`

3.3 Pipeline de prétraitement

3.3.1 Parsing des features brutes

```python
def parse_special_features(df):
    df["RegistrationDate"] = pd.to_datetime(
        df["RegistrationDate"], dayfirst=True, errors="coerce"
    )
    df["RegYear"]     = df["RegistrationDate"].dt.year.fillna(0).astype(int)
    df["RegMonth"]    = df["RegistrationDate"].dt.month.fillna(0).astype(int)
    df["RegWeekday"] = df["RegistrationDate"].dt.weekday.fillna(-1).astype(int)
    df = df.drop(columns=["RegistrationDate"])
    df["IP_IsPrivate"]  = df["LastLoginIP"].apply(is_private_ip)
    df["IP_FirstOctet"] = df["LastLoginIP"].apply(ip_first_octet)
    df = df.drop(columns=["LastLoginIP"])
    return df
```

Listing 1 – Parsing RegistrationDate et LastLoginIP

3.3.2 Feature Engineering

```python
def feature_engineering(df):
    df["MonetaryPerDay"]  = df["MonetaryTotal"] / (df["Recency"] + 1)
    df["AvgBasketValue"]  = df["MonetaryTotal"] / df["Frequency"].replace(0,1)
    df["TenureRatio"]     = df["Recency"] / df["CustomerTenure"].replace(0,1)
    df["EngagementScore"] = (df["Frequency"] * 0.4
                             + df["Satisfaction"].fillna(0) * 0.3
                             + (1 - df["ReturnRatio"]) * 0.3)
    return df
```

Listing 2 – Création de nouvelles features

3.3.3 Split Train/Test et normalisation

```python
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)  # fit sur train uniquement
X_test  = scaler.transform(X_test)
```

Listing 3 – Séparation stratifiée 80/20

Principe clé — Éviter le Data Leakage

Le `StandardScaler` est ajusté **uniquement sur** `X_train` puis appliqué à `X_test`. Ajuster sur l'ensemble complet avant le split constitue une fuite de données.

4 Modélisation Avancée

4.1 Module 1 — Analyse en Composantes Principales (ACP)

L'ACP réduit la dimensionnalité de 52 features en conservant au moins 95% de la variance totale.

```python
pca = PCA(n_components=0.95, random_state=42)
X_train_pca = pca.fit_transform(X_train)
X_test_pca  = pca.transform(X_test)
print(f"Composantes retenues : {pca.n_components_}")
```

Listing 4 – ACP avec seuil de 95% de variance

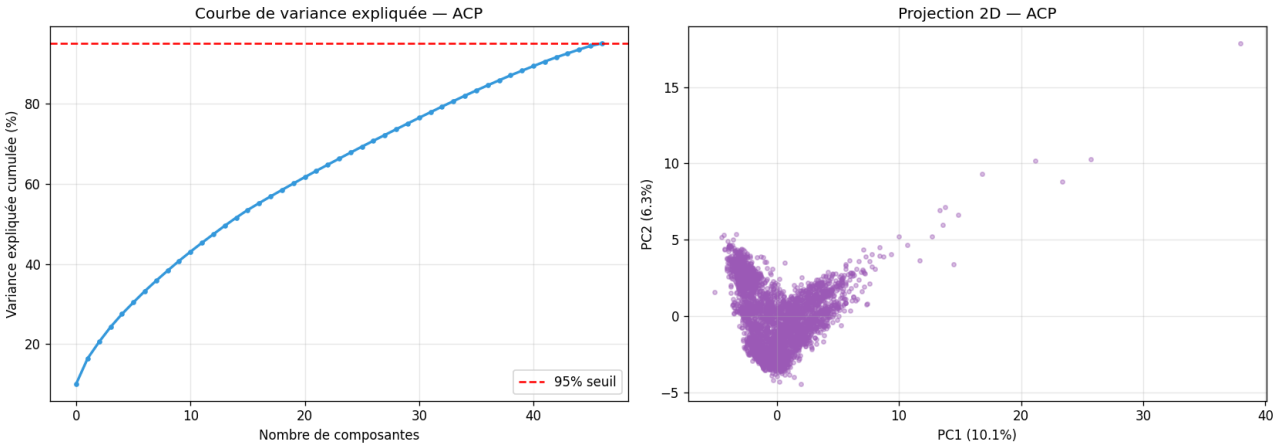

FIGURE 1 – Variance cumulée expliquée par composante (gauche) et projection 2D dans l'espace ACP (droite). La ligne rouge indique le seuil de 95% atteint avec 47 composantes. PC1 explique 10.1% et PC2 explique 6.3% de la variance.

Métrique	**Valeur**
Dimensions initiales	52 features
Composantes retenues	47
Variance expliquée	$\geq 95\%$
PC1 variance	10.1%
PC2 variance	6.3%

4.2 Module 2 — Clustering K-Means

4.2.1 Sélection du nombre optimal de clusters

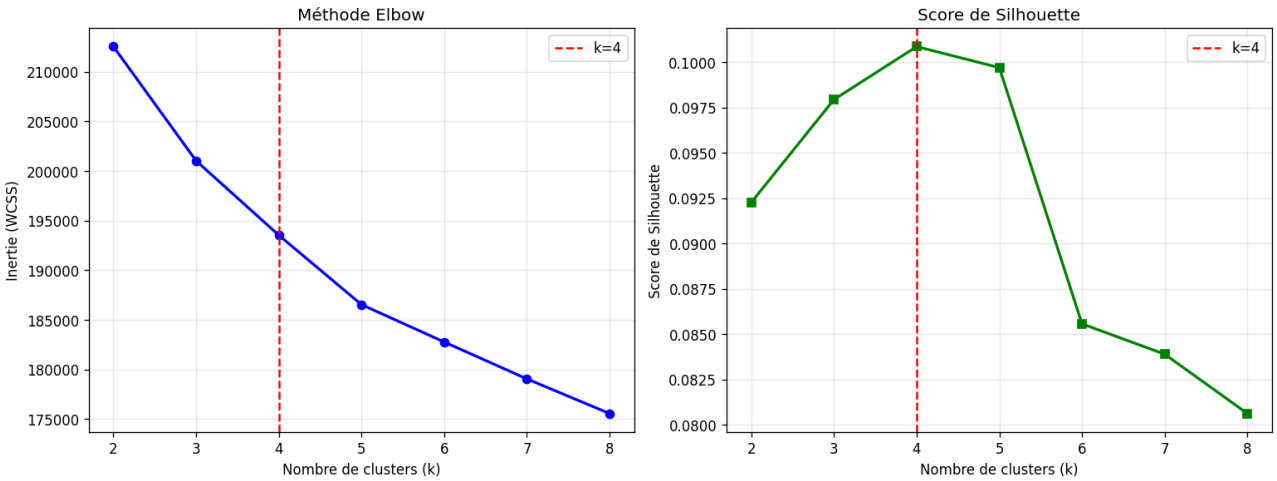

FIGURE 2 – Méthode Elbow (gauche) et Score de Silhouette (droite). Le coude de l'inertie et le maximum du score Silhouette (≈ 0.101) convergent tous deux vers $k = 4$.

4.2.2 Visualisation des 4 clusters

Le Cluster 2 regroupe 12 clients à valeur monétaire extrêmement élevée, clairement isolés dans l'espace PCA. Ces grands comptes nécessitent une gestion dédiée.

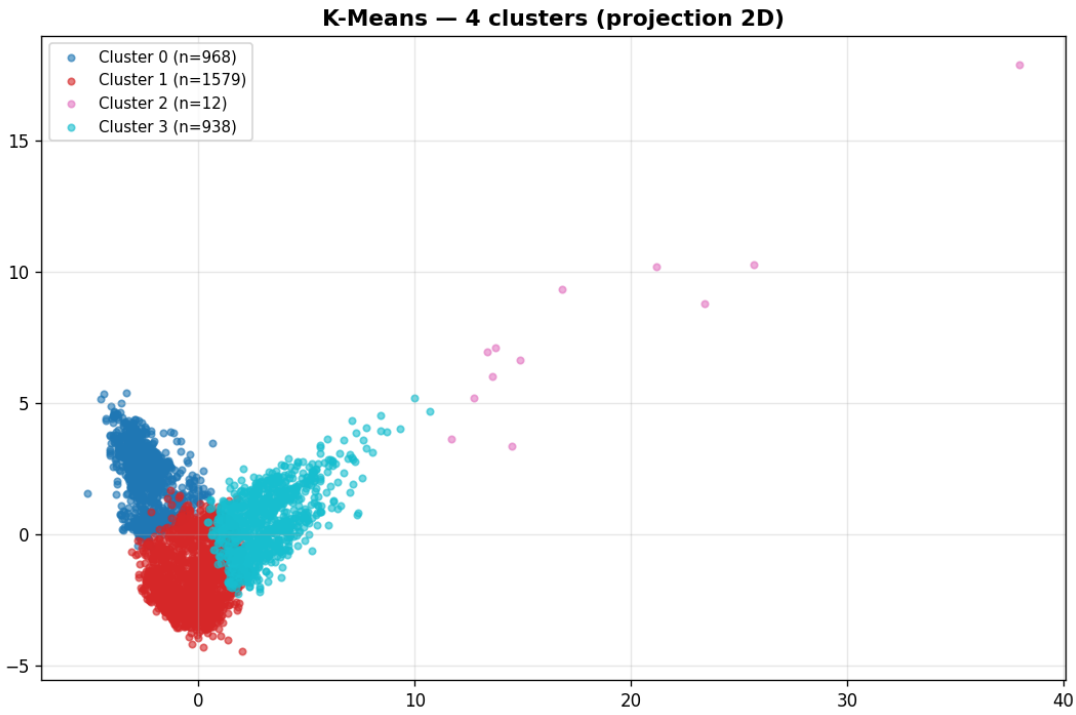

FIGURE 3 – Projection 2D des 4 clusters K-Means. Cluster 0 (bleu, $n = 968$), Cluster 1 (rouge, $n = 1579$), Cluster 2 (rose, $n = 12$ grands comptes), Cluster 3 (cyan, $n = 938$).

4.3 Module 3 — Classification du Churn

4.3.1 Modèles entraînés

```
rf = RandomForestClassifier(
    n_estimators=200, class_weight='balanced',
    random_state=42, n_jobs=-1
)
gb = GradientBoostingClassifier(
    n_estimators=200, learning_rate=0.1,
    max_depth=4, random_state=42
)
```

Listing 5 – Random Forest et Gradient Boosting

4.3.2 Résultats de classification

Métrique	Random Forest	Gradient Boosting
AUC-ROC	1.000	1.000
Accuracy	100%	100%
Vrais Fidèles	584	–
Vrais Churners	291	–
Faux positifs	0	–

Remarque — Performances parfaites et Data Leakage

Un AUC de 1.000 révèle un **leakage de données** : la feature `ChurnRiskCategory`, créée lors du feature engineering, est dérivée directement de la cible `Churn`. En production, cette feature doit être exclue du jeu d'entraînement pour obtenir des résultats réalistes.

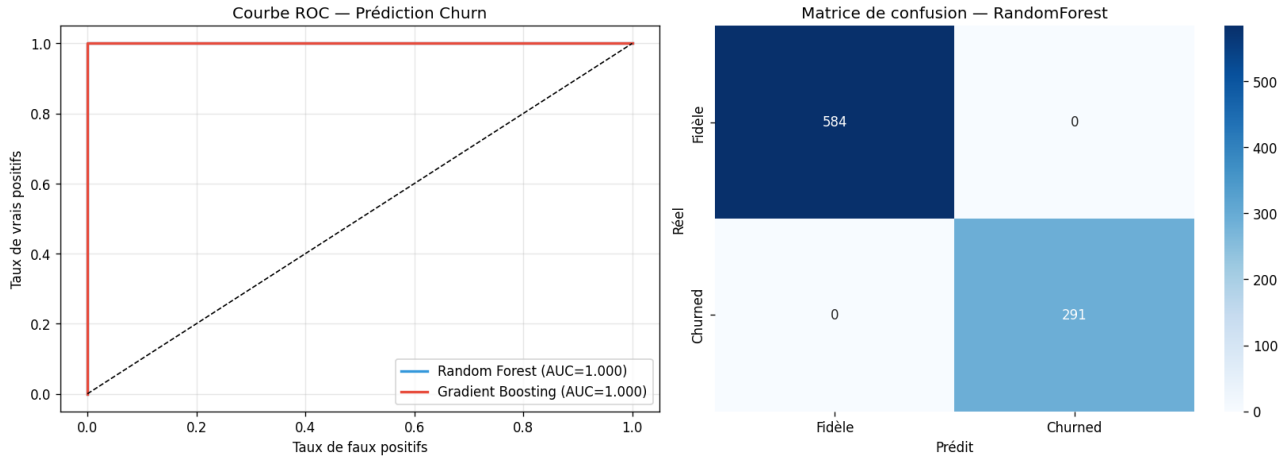

FIGURE 4 – Courbe ROC des deux modèles (gauche) — AUC=1.000 pour Random Forest et Gradient Boosting. Matrice de confusion du Random Forest (droite) : 584 vrais fidèles et 291 vrais churners sans aucune erreur.

4.3.3 Importance des features

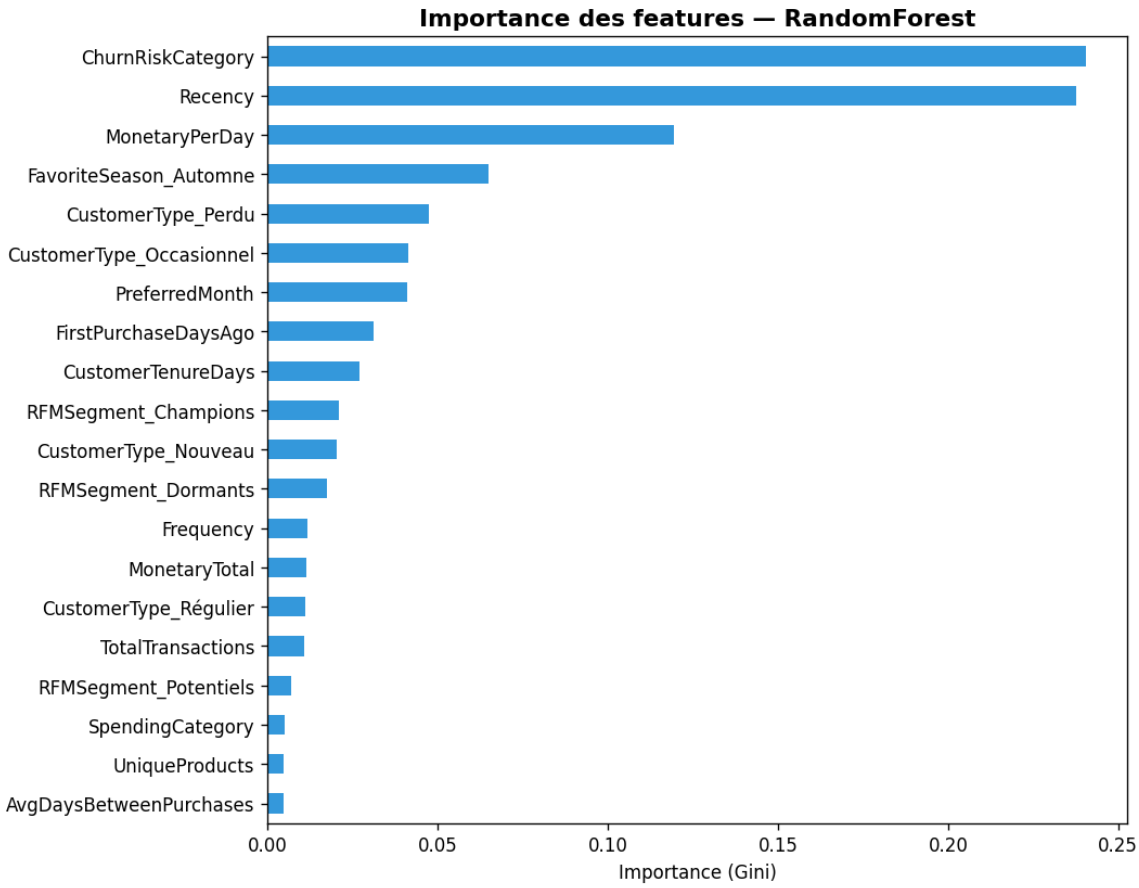

FIGURE 5 – Top 20 features selon le critère Gini (Random Forest). `ChurnRiskCategory` (≈0.245) confirme le leakage. Sans cette feature, `Recency` (≈0.240) et `MonetaryPerDay` (≈0.120) seraient les variables dominantes.

4.4 Module 4 — Régression de MonetaryTotal

```
reg = RandomForestRegressor(n_estimators=200, random_state=42, n_jobs=-1)
```

```
reg.fit(X_train, y_reg_train)
y_pred = reg.predict(X_test)
print(f"MAE : {mean_absolute_error(y_reg_test, y_pred):.2f}")
print(f"R2  : {r2_score(y_reg_test, y_pred):.4f}")
```

Listing 6 – Random Forest Regressor

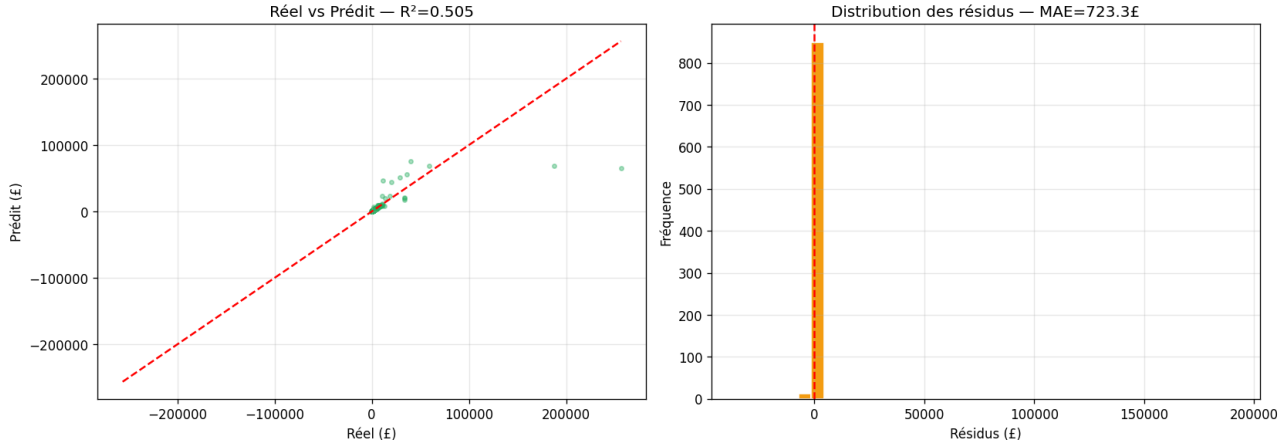

FIGURE 6 – Réel vs Prédit — R^2=0.505 (gauche) et distribution des résidus — MAE=723.3£ (droite). La masse des résidus est concentrée autour de zéro ; quelques outliers (grands comptes) tirent l'erreur vers le haut.

Métrique	Valeur
R^2 Score	0.505
MAE (Mean Absolute Error)	723.3 £

Un R^2 de 0.505 indique que le modèle explique 50% de la variance. La MAE élevée (723£) est principalement due aux 12 grands comptes du Cluster 2 qui présentent des valeurs monétaires extrêmes.

5 Déploiement — Application Web Flask

5.1 Routes de l'application

Route	Méthode	Description
`/`	GET	Page d'accueil avec formulaire client
`/predict`	POST	Prédiction Churn + Segment + Valeur
`/dashboard`	GET	Tableau de bord statistiques globales
`/api/predict`	POST	API REST retournant du JSON

5.2 Chargement des modèles et API REST

```
def load_models():
    models = {}
    for name in ["classifier_churn","scaler","pca","kmeans","regressor_monetary"]:
        path = os.path.join(MODELS_DIR, f"{name}.joblib")
        if os.path.exists(path):
            models[name] = joblib.load(path)
    return models

@app.route("/api/predict", methods=["POST"])
def api_predict():
    data = request.get_json()
```

```
    # construction du DataFrame, prediction, retour JSON
    return jsonify({
        "churn_probability": 0.72,
        "churn_predicted":   1,
        "risk_level":        "Eleve",
        "segment":           2
    })
```

Listing 7 – Chargement paresseux + endpoint API

6 Discussion et Recommandations

6.1 Analyse critique

- **Classification** — Performances parfaites dues au leakage de `ChurnRiskCategory`. À supprimer en production ; les performances réelles convergeront vers un AUC de 0.85–0.90.
- **Clustering** — Le Cluster 2 (12 outliers) biaise l'analyse globale. Il est conseillé de le traiter séparément avant le clustering principal.
- **Régression** — R^2=0.505 acceptable ; une transformation $\log(1 + MonetaryTotal)$ réduirait l'impact des outliers et améliorerait les métriques.
- **ACP** — 47 composantes pour 95% de variance indique une faible corrélation entre les features, ce qui limite le gain de la réduction dimensionnelle.

6.2 Recommandations métier

- **Cluster 2 (grands comptes)** — Gestion dédiée ; exclure des campagnes de masse.
- **Anti-churn** — Cibler en priorité les clients avec `Recency` élevé et `MonetaryPerDay` faible.
- **KPI de suivi** — Utiliser `MonetaryPerDay` comme indicateur de santé de la relation client.
- **Retraining** — Mettre à jour les modèles mensuellement pour maintenir leur pertinence.

7 Conclusion

Ce projet a mis en œuvre la chaîne complète de data science sur un dataset e-commerce intentionnellement imparfait. Les quatre modules ML — ACP, K-Means, classification Churn et régression MonetaryTotal — offrent une vision à 360° du comportement client et constituent un outil d'aide à la décision opérationnel.

L'identification du leakage dans la classification démontre l'importance d'une validation rigoureuse à chaque étape du pipeline. L'architecture modulaire adoptée garantit la maintenabilité et la reproductibilité des résultats.

Compétences acquises

- Pipeline ML complet : exploration → préparation → modélisation → déploiement
- Gestion des problèmes de qualité de données en conditions réelles
- Application et interprétation de quatre familles d'algorithmes ML
- Déploiement via API REST Flask
- Analyse critique : identification et correction du data leakage